Database
(SOC-3020)

Alessandro Agostini[1,2]

aagostini@inha.ac.kr
agostini@disi.unitn.it

[1]LDKR-Group, School of Computer and Information Engineering
Inha University in Tashkent (IUT), Uzbekistan

[2]DISI, University of Trento, Italy

WEEK 1





DATABASE

Database Systems (DBMSs): Overview

WEEK 1

Code: W1.L1L2

TODAY'S AGENDA

1. Definition of DBMSs
2. DBMSs' Applications and Purposes
3. DBMSs' Architecture and Users
4. Before were File Systems...
5. View of Data, or: Levels of Data Abstraction
6. Data Models & Languages
7. Very Brief History of DBMSs





Database Management System (DBMS)

- **Def**: A database-management system (DBMS) is a collection of interrelated data (i.e, information) about a particular enterprise together with a set of programs to access those data.
- So:
  - Collection of interrelated data
  - Set of programs to access the data
- A DBMS provides an environment that is both *convenient* and *efficient* to use.





DBMS - Applications

- Database Applications:
  - Banking: transactions
  - Airlines: reservations, schedules
  - Universities: registration, grades
  - Sales: customers, products, purchases
  - Online retailers: order tracking, customized recommendations
  - Manufacturing: production, inventory, orders, supply chain
  - Human resources: employee records, salaries, tax deductions
  - …



Outline

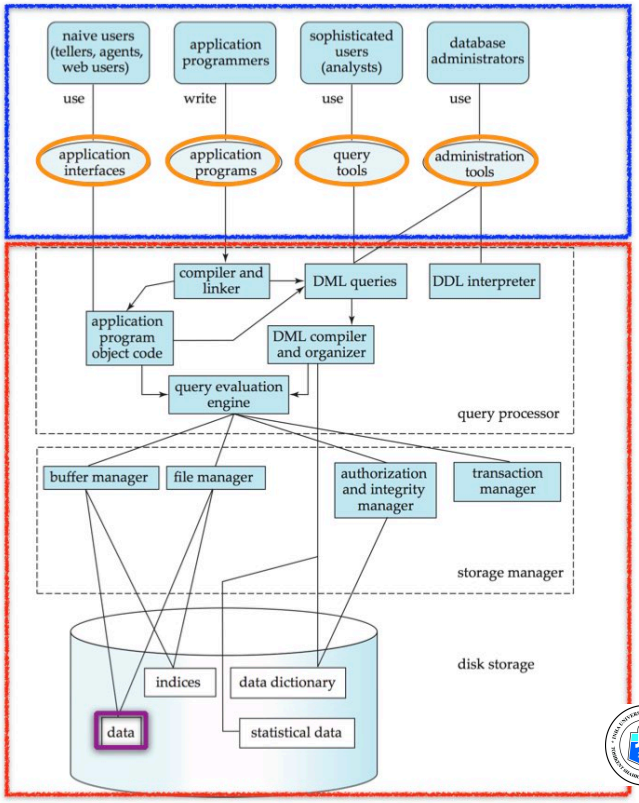
1. Definition of DBMSs
2. DBMSs' Applications and Purposes
3. DBMSs' Architecture and Users
4. Before were File Systems...
5. View of Data, or: Levels of Data Abstraction
6. Data Models & Languages
7. Very Brief History of DBMSs



DBMS Architecture

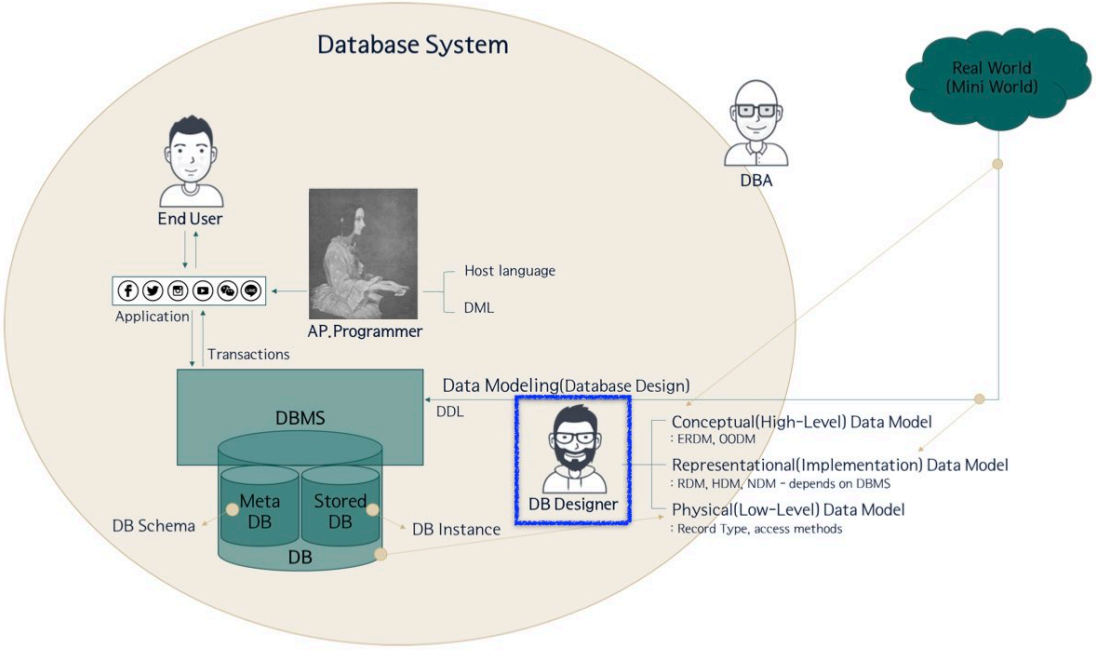
users and administrators

internal components





Database Users



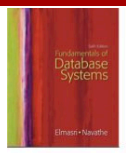




DBMS Architecture (*cont.*)

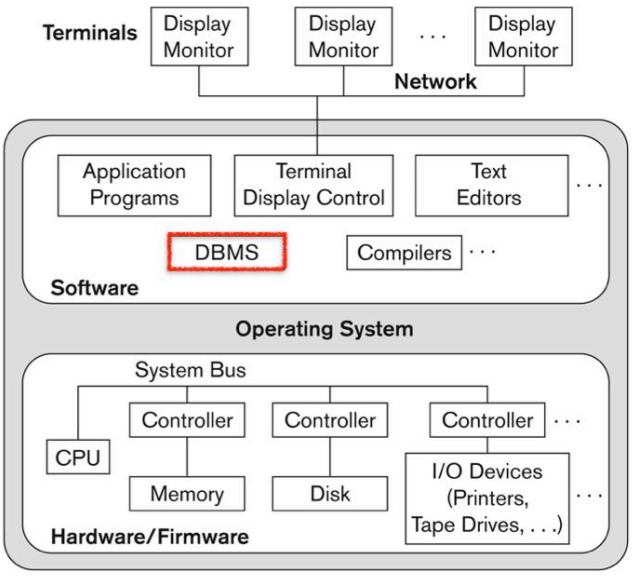
- Greatly influenced by underlying computer system on which the database is running:
  - Centralized
  - Client-server
  - Parallel (multi-processor)
  - Distributed





Centralized DBMS Architecture

- Example of physical centralized architecture:



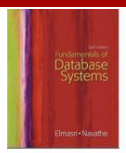
Note:
PCs have their own display/ monitor

no network needed

So, common case of centralized DBMS architecture

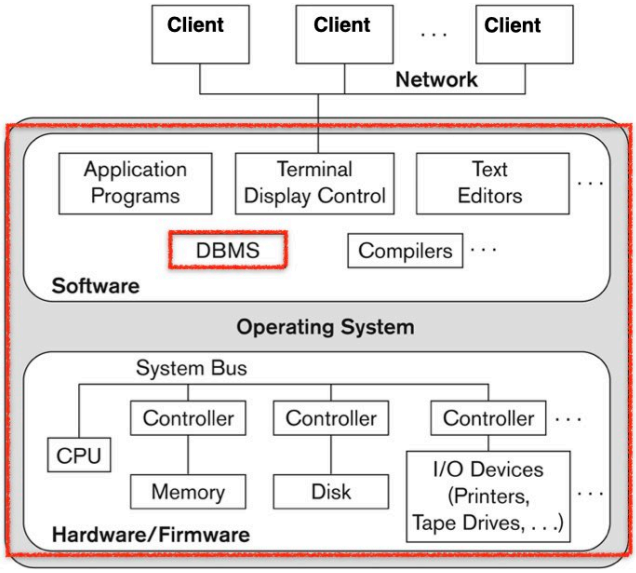
cfr. Reference book (R1)





Client-Server DBMS Architecture

- Example of client-server (physical) architecture:



cfr. Reference book (R1) (*modified*)

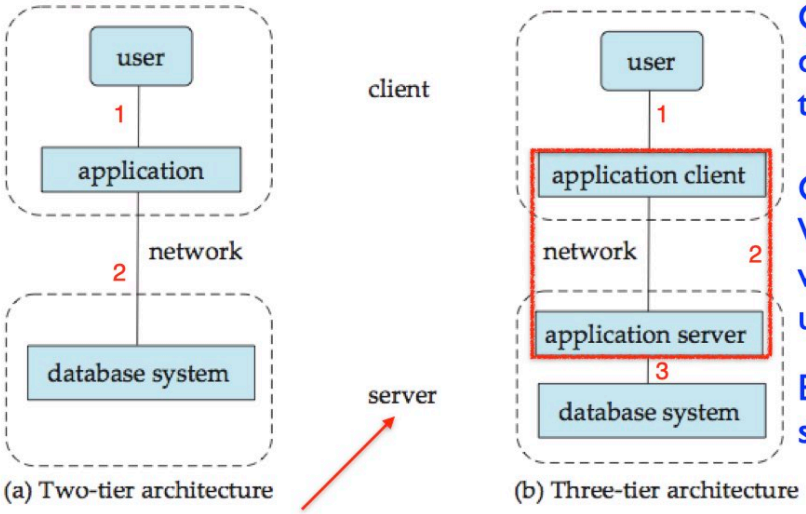


Client-Server DBMS Architectures (*cont.*)

- DBMS Server relates to Client/User in two ways:

Clients can *directly access* to DB server (but also Printer/File/ Mail…servers).

Access via API standards ODBC/JDBC.



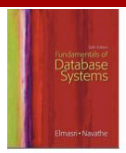
(a) Two-tier architecture (b) Three-tier architecture

Clients can't directly *access* to DB server.

Common for Web/systems with many users.

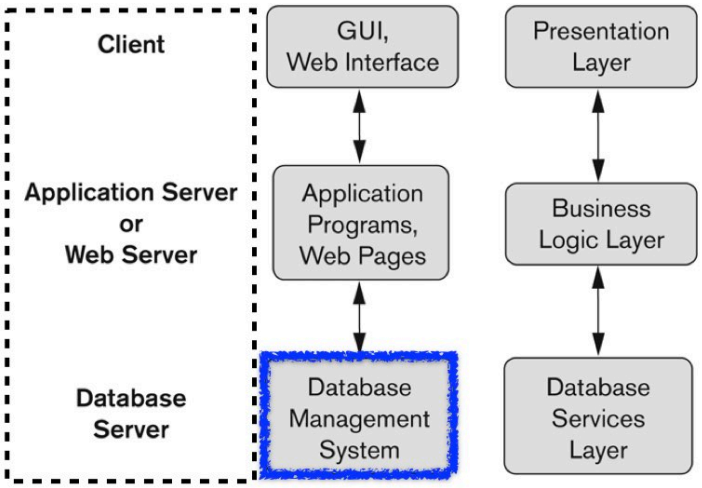
Enhance security.

Terminology: called SQL server if DBMS is Relational



Client-Server DBMS Architectures (*cont.*)

- Three-tier DBMS architecture (more terminology):



cfr. Reference book (R1)



Outline

1 Definition of DBMSs

2 DBMSs' Applications and Purposes

3 DBMSs' Architecture and Users

4 Before were File Systems...

5 View of Data, or: Levels of Data Abstraction

6 Data Models & Languages

7 Very Brief History of DBMSs

Before DBMSs were files...

or: On Limits of File Systems to Manage Data



Database Systems as File Systems

- In the early days, database applications were built directly on top of file systems (O.S.'s)
- **Question**:
  What is a today example of "database system" based on files? … (*we use it every day, maybe*)
- **Answer**: ?





Database Systems as File Systems

- In the early days, database applications were built directly on top of file systems (O.S.'s)
- **Question**:
  What is a today example of database system based on files? … (*we use it every day, maybe*)
- **Answer**: MS Excel / Modern OSs (*if search allowed*)





Drawbacks of Using File Systems

- **Data redundancy and inconsistency**
  - Multiple file formats, duplication of information in different files





Drawbacks of Using File Systems

- **Data redundancy and inconsistency**
  - Multiple file formats, duplication of information in different files
- **Difficulty in accessing data** (*query processing*)
  - Need to write a new application program to carry out each new task over available data
  - **Example**: ?







Drawbacks of Using File Systems

- **Data redundancy and inconsistency**
  - Multiple file formats, duplication of information in different files
- **Difficulty in accessing data** (*query processing*)
  - Need to write a new application program to carry out each new task over available data
  - **Example**:
    Poor selectivity of "search" in PCs; you can access by "keyword" only: *all* docs with that keyword (and using an additional keyword could not be satisfactory for certain tasks)



Drawbacks of Using File Systems



Spring, Summer, Winter And Fall
Aphrodite's Child
Best Of Aphrodite's Child

It's a nice song, by the way…

- **Example** (*do search for "Fall semester"…*): Poor selectivity of "search" in PCs; you can access by "keyword" only: *all* docs with that keyword (and using an additional keyword could not be satisfactory for certain tasks)





Drawbacks of Using File Systems

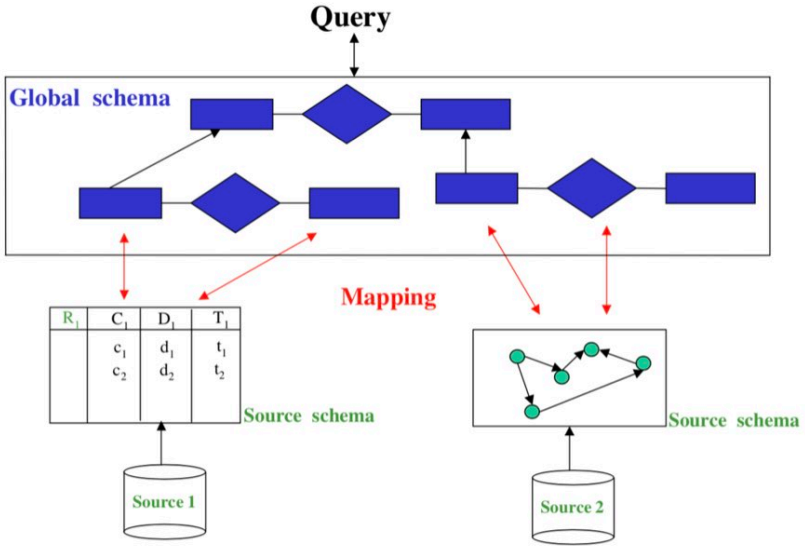
- **Data redundancy and inconsistency**
  - Multiple file formats, duplication of information in different files
- **Difficulty in accessing data** (*query processing*)
  - Need to write a new application program to carry out each new task over available data
- **Data isolation**
  - Multiple files and formats; no links among files
- **Integrity problems**
  - Hard to add new constraints or change existing ones (E.g., numerical constraint in MS Excel.)

Data Integration Problem



Data Integration

E.g.:
Knowledge Graph



Source: Lenzerini, PODS 2002





Drawbacks of Using File Systems

- **Atomicity of updates**
  - Failures may leave database in an inconsistent state with partial updates carried out
  - **Example**: ?







Drawbacks of Using File Systems

- **Atomicity of updates**
  - Failures may leave database in an inconsistent state with partial updates carried out
  - **Example**:
    Transfer of funds from one account to another should either complete or not happen at all.





Drawbacks of Using File Systems

- **Concurrent access by multiple users**
  - Concurrent access needed for performance
  - Uncontrolled concurrent accesses can lead to inconsistencies
  - **Example**: ?







Drawbacks of Using File Systems

- **Concurrent access by multiple users**
  - Concurrent access needed for performance
  - Uncontrolled concurrent accesses can lead to inconsistencies
  - **Example**:
    Two people reading a balance and updating it by withdrawing money at the same time.





Drawbacks of Using File Systems

- **Security problems**
  - Hard to provide user access to some, but not all, data
  - **Example**:
    Password on Acrobat Reader for PDFs is for all the content, not *some* of it.



To solve all drawbacks above...

...DBMSs have been developed since mid 1960s!

https://amturing.acm.org/award_winners/bachman_9385610.cfm

1973 ACM Turing Award Lecture

The Turing Award citation read by Richard G. Canning, chairman of the 1973 Turing Award Committee, at the presentation of this lecture on August 28 at the ACM Annual Conference in Atlanta:

A significant change in the computer field in the last five to eight years has been made in the way we treat and handle data. In the early days of our field, data was intimately tied to the application programs that used it. Now we see that we want to break that tie. We want data that is independent of the application programs that use it—that is, data that is organized and structured to serve many applications and many users. What we seek is the *data base*.

This movement toward the data base is in its infancy. Even so, it appears that there are now between 1,000 and 2,000 true data base management systems installed worldwide. In ten years very likely, there will be tens of thousands of such systems. Just from the quantities of installed systems, the impact of data bases promises to be huge.

This year's recipient of the A.M. Turing Award is one of the real pioneers of data base technology. No other individual has had the influence that he has had upon this aspect of our field. I single out three prime examples of what he has done. He was the creator and principal architect of the first commercially available data base management system—the Integrated Data Store—originally developed from 1961 to 1964.[1,2,3,4] I-D-S is today one of the three most widely used data base management systems. Also, he was one of the founding members of the CODASYL Data Base Task Group, and served on that task group from 1966 to 1968. The specifications of that task group are being implemented by many suppliers in various parts of the world.[5,6] Indeed, currently these specifications represent the only proposal of stature for a common architecture for data base management systems. It is to his credit that these specifications, after extended debate and discussion, embody much of the original thinking of the Integrated Data Store. Thirdly, he was the creator of a powerful method for displaying data relationships—a tool for data base designers as well as application system designers.[7,8]

His contributions have thus represented the union of imagination and practicality. The richness of his work has already had, and will continue to have, a substantial influence upon our field.

I am very pleased to present the 1973 A.M. Turing Award to Charles W. Bachman.

CHARLES WILLIAM BACHMAN

United States – 1973

CITATION

For his outstanding contributions to database technology.



The Programmer as Navigator

by Charles W. Bachman

Outline

1 Definition of DBMSs

2 DBMSs' Applications and Purposes

3 DBMSs' Architecture and Users

4 Before were File Systems...

5 View of Data, or: Levels of Data Abstraction

6 Data Models & Languages

7 Very Brief History of DBMSs



Reminder

- A database (management) system is a "sum" of
  - **interrelated data**
    and a
  - **set of programs**
    that allow users to access and modify (*conveniently, efficiently*) these data.

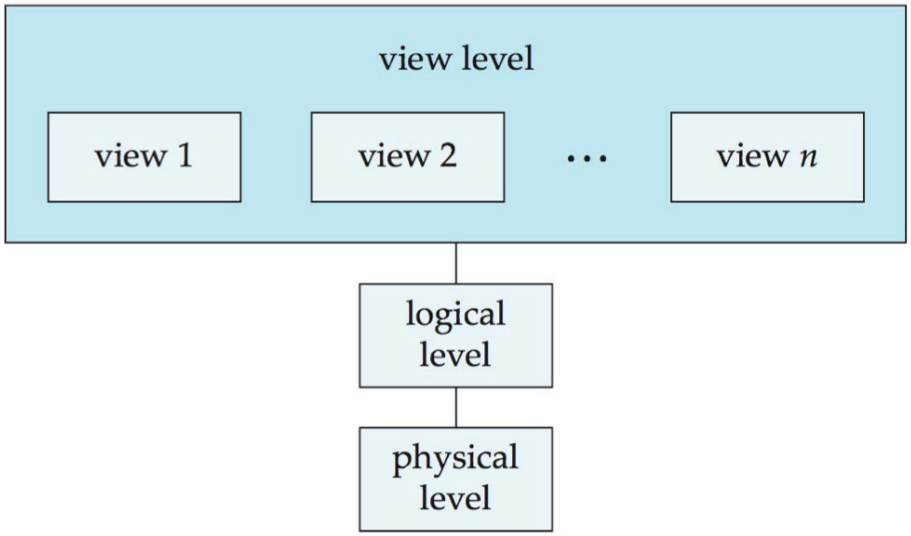
- A purpose of a database (management) system is to provide users with an *abstract* view of data.

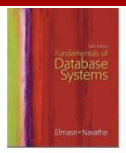




View of Data

An architecture for a database system:



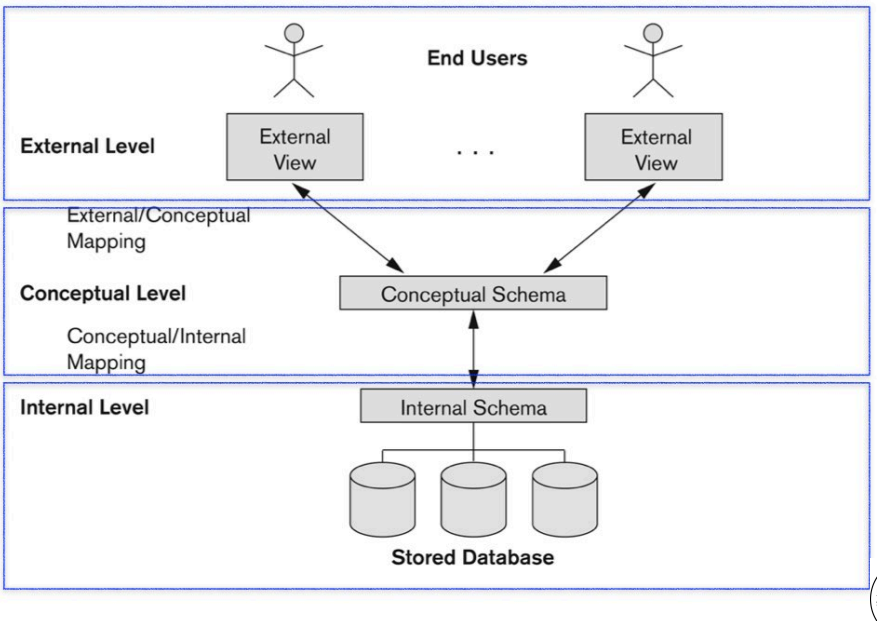


View of Data (*variant*)

View Level:

Conceptual / Logical Levels:

Physical Level:



cfr. Reference book (R1)

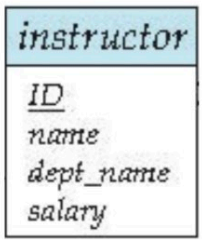


Levels of Abstraction

- **Physical level:** describes how a record (e.g., see *instructor* below) is physically stored.
- **Logical level:** describes what data are stored in database, and the relationships among the data.

Example:

```
type instructor = record
        ID : string;
        name : string;
        dept_name : string;
        salary : integer;
    end;
```



- **View level:** application programs hide details of data types. Views can also hide information (such as an employee's salary or student's credits) for security/privacy purposes.





Data Independence

- **Logical Data Independence** – property of DBMSs to allow change of conceptual schema (*logical level*) without having to change the external schemas (*view level*) and their associated application programs.
  - **Note**: Applications (i.e., user programs) depend on / are designed to deal with the View Level.

- **Physical Data Independence** – property of DBMSs to allow change of the internal schema (*physical level*) without having to change the conceptual/logical schema to access data.





Instances and Schemas

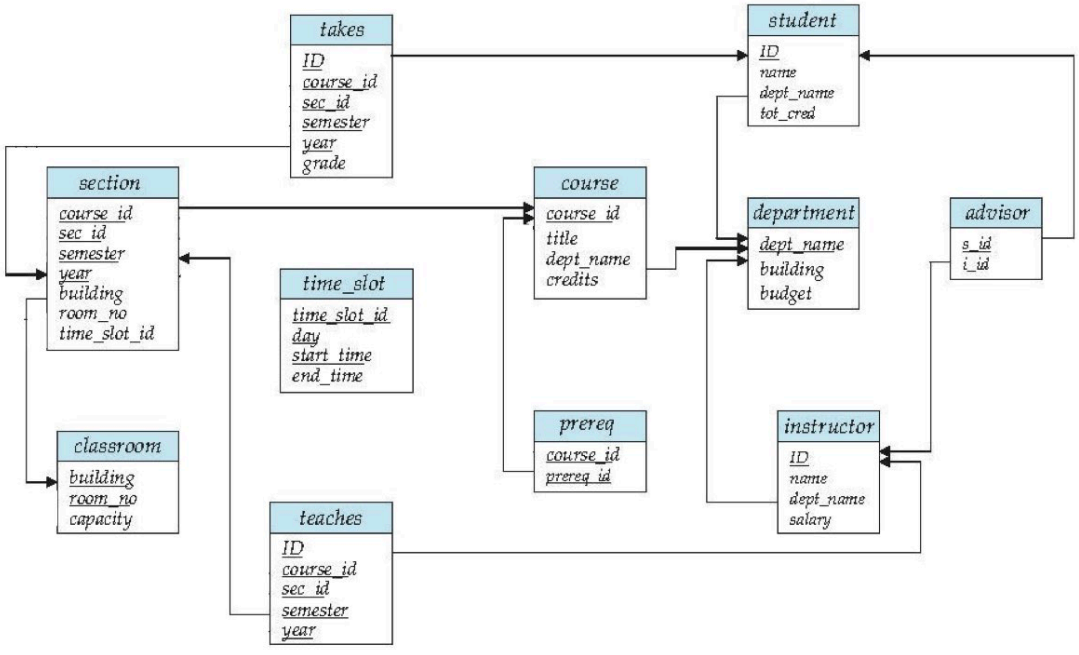
- **Schema**– the overall structure of the database
  - Analogous to the type of a variable in a program
- **Instance** – the content of the database at a particular point in time
  - Analogous to the value of a variable

ID	name	salary	dept_name	building	budget
22222	Einstein	95000	Physics	Watson	70000
12121	Wu	90000	Finance	Painter	120000
32343	El Said	60000	History	Painter	50000
45565	Katz	75000	Comp. Sci.	Taylor	100000
98345	Kim	80000	Elec. Eng.	Taylor	85000
76766	Crick	72000	Biology	Watson	90000
10101	Srinivasan	65000	Comp. Sci.	Taylor	100000
58583	Califieri	62000	History	Painter	50000
83821	Brandt	92000	Comp. Sci	Taylor	100000
15151	Mozart	40000	Music	Packard	80000
33456	Gold	87000	Physics	Watson	70000
76543	Singh	80000	Finance	Painter	120000





Schema: Example





Outline

1. Definition of DBMSs
2. DBMSs' Applications and Purposes
3. DBMSs' Architecture and Users
4. Before were File Systems...
5. View of Data, or: Levels of Data Abstraction
6. Data Models & Languages
7. Very Brief History of DBMSs

DBMSs

Data Models & Languages



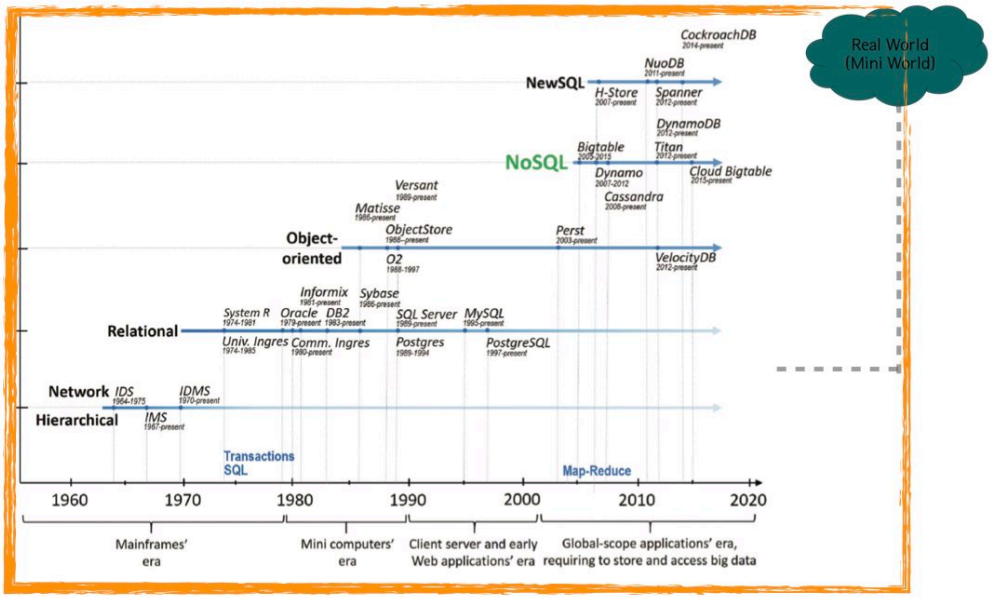
Data Models

- **Def**: A collection of conceptual tools for describing:
  - Data and Data relationships
  - Data semantics
  - Data constraints (consistency, duplications, etc)
- Relational model
- Entity-Relationship data model (mainly for DB design)
- Object-based data models (Object-oriented and Object-relational)
- Semistructured data model (XML / RDF / JSON …)
- Other models:
  - Network model / Hierarchical model / NoSQL …



Data(base) *Systems*

[Davoudian et al., 2018]





Data Models



(Blumauer & Nagy, 2020)

Knowledge
Information
Data

The **World** is…
"the totality of Facts" (Wittgenstein)
i.e.,
.
.
.
Data!

- Data and Data relationships
- Data semantics
- Data constraints (*internal*, business rules, …)





DBMSs Languages

- Database systems provide a set of linguistic tools to specify everything about the data it deals with:
  - Data-Definition Language (**DDL**): to specify the (logical, physical) schema
  - Data-Manipulation Language (**DML**): to specify the operations to manage the (instances of) data, e.g.:
    - Querying (information retrieval)
    - Inserting new data
    - Deleting existing data
    - Modifying existing data …

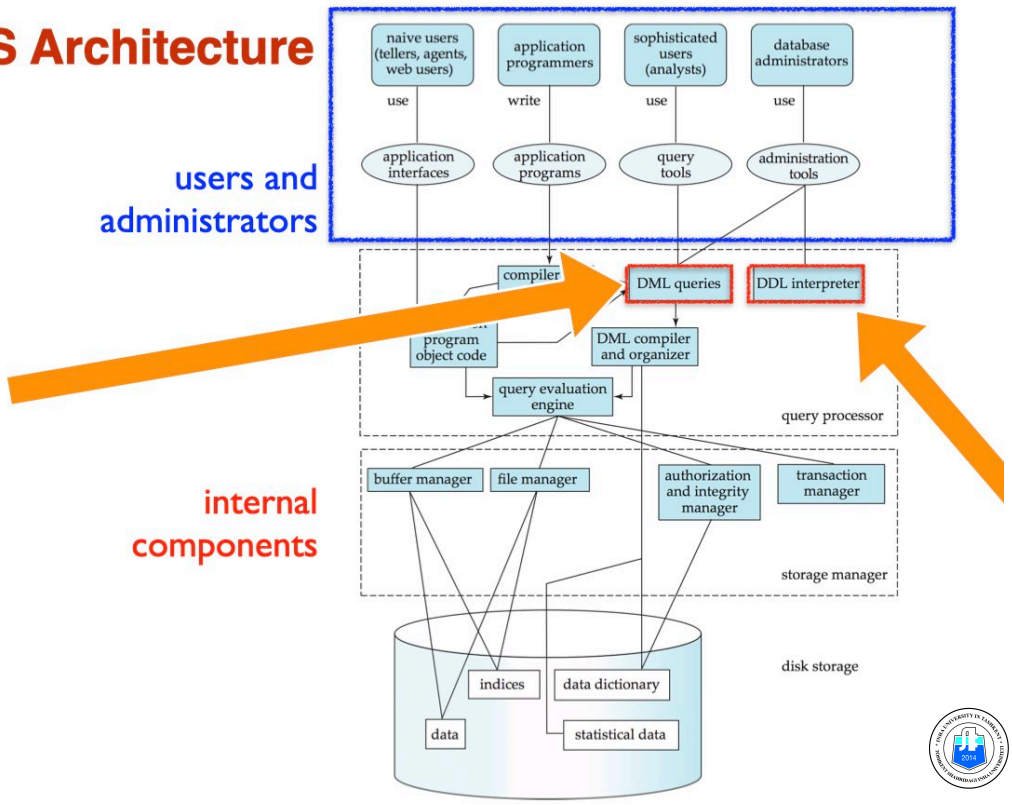




DBMS Architecture

users and administrators

internal components





Relational Query Languages

- "Pure" relational query languages:
  - Relational algebra (*procedural*)
  - Tuple relational calculus (*non-procedural*)
  - Domain relational calculus (*non-procedural*)
- These languages are equivalent in computing power and expressiveness.
- SQL (more expressive; based on pure languages)



Outline

1. Definition of DBMSs
2. DBMSs' Applications and Purposes
3. DBMSs' Architecture and Users
4. Before were File Systems...
5. View of Data, or: Levels of Data Abstraction
6. Data Models & Languages
7. Very Brief History of DBMSs

DBMSs

(Very) Brief History



Brief History of Database Systems

- 1950s and early 1960s:
  - Data processing using magnetic tapes for storage
    - Tapes provided only sequential access
  - Punched cards for input
- Late 1960s and 1970s:
  - Hard disks allowed direct access to data
  - Network and hierarchical data models in widespread use
  - Codd [1970] defines the Relational Data Model
    - Would win the ACM Turing Award for this work
    - IBM Research begins System R prototype
    - UC Berkeley begins Ingres prototype
    - (1979) First RDBMS (by RSI, i.e., Oracle since 1983)
  - High-performance (for the era) transaction processing



Brief History (*cont.*) - Main Reference

Suggested reading (non-technical parts):

```
@Article{Codd70cacm,
  author    = "E. F. Codd",
  title     = "A Relational Model of Data for Large Shared Data Banks",
  journal   = "Communications of the ACM",
  year      = "1970",
  volume    = "13",
  number    = "6",
  pages     = "377--387"
}
```





Brief History (*cont.*)

- 1980s:
  - Research relational prototypes evolve into commercial systems - SQL becomes industrial standard
  - Parallel and distributed database systems
  - Object-oriented database systems
- 1990s:
  - Large decision support and data-mining applications
  - Large multi-terabyte data warehouses
  - Emergence of Web commerce
- Early 2000s:
  - XML and XQuery standards
  - Automated database administration; Semantic Web
- Later 2000s:
  - Giant data storage systems: Google, Yahoo, Amazon, …



Summary

1. Definition of DBMSs
2. DBMSs' Applications and Purposes
3. DBMSs' Architecture and Users
4. Before were File Systems...
5. View of Data, or: Levels of Data Abstraction
6. Data Models & Languages
7. Very Brief History of DBMSs



DATABASE

End of Slides

WEEK 1

Code: W1.L1L2